



Technical notes

MAPping **AL**loys: A Python library for
mapping features and properties of alloys
over compositional spaces

Table of Contents

1. Introduction to MAPAL	1
1.1. Important user notes	1
1.2. How to cite	2
2. Installation.....	3
3. Quick notes on packages	4
3.1. MAPAL in-built packages	4
3.2. Integrating MAPAL with CoSMoR	4
4. Packages.....	5
4.1. <i>mapal.element_data</i>	5
4.2. <i>mapal.create_alloys</i>	8
4.3. <i>mapal.alloy_features</i>	14
4.4. <i>mapal.miedema_calc</i>	24
4.5. <i>mapal.preTrained_ML_mods</i>	26
4.5.1 References for pre-trained ML models included in MAPAL:	30
4.6. <i>mapal.map_features</i>	31
4.7. <i>mapal.map_ML_predictions</i>	37

1. Introduction to MAPAL

A preliminary conceptualization of the alloy design process often requires an extension of elemental properties to compositional spaces. A classic example is the Hume-Rothery rules that use elemental properties to describe the approximate conditions under which solid solutions might form in binary alloys. Moreover, in recent years, various machine learning models have used features derived from elemental properties to predict structure and properties of alloys.

The primary goals behind the development of 'MAPAL' are to enable:

1. Easy mapping of alloy features over compositional variations in binary, ternary and multi-principal element alloy (MPEA) systems.
2. Integration with machine learning frameworks that require calculation of alloy features.
3. Use of pre-trained machine learning models to map phase selection and properties over compositional spaces in MPEAs.

1.1. Important user notes

1. Alloy representation: MAPAL uses a '*string*' format for alloy name to capture the composition of any given alloy. A numeric suffix (*'integer'* or *'float'*) after the element represents its stoichiometry within the alloy. If an element does not have a suffix, stoichiometry of '1' is considered. This terminology is in line with the commonly adopted representation for MPEAs. Some examples are as below:

- 'AlTi' represents equiatomic binary of Al & Ti; whereas 'Al₂Ti' or 'AlTi_{0.5}' is a binary alloy where number of atoms of Al:Ti are in a 2:1 ratio.
- 'CoCrFeNi' is an equiatomic quaternary; whereas 'CoCr₂FeNi_{2.5}' is a quaternary alloy where number of atoms of Co:Cr:Fe:Ni are in a 1:2:1:2.5 ratio.

2. Operators, mutable element properties and in-built features: MAPAL generates alloy features based on two methodologies:

- *Using an operator to mutate elemental properties:* An operator is a mathematical function that can be applied to any mutable property to create composition dependent alloy feature. For e.g., 'asymmetry' operator can be applied to any mutable element property such as 'radius', 'modulus', and 'cohesive energy'. Creation of such features requires passing a '*tuple*' specifying both the operator and mutable element property, for e.g., ('asymmetry', 'r_{met}') or ('asymmetry', 'E')
- *Creating an in-built feature:* A feature based on defined function. For e.g., 'configurational entropy', 'lattice distortion energy' or 'Wang parameter'. While an operator can be applied to any elemental property, in-built features are strictly defined parameters and thus require

passing only a 'string' specifying the feature, for e.g., 'S_config', 'latt_dist_energy' or 'Wang_parameter'.

Refer to documentation of ['mapal.alloy_features'](#) package for more information on all the operators and features available.

3. Components: While creating compositional spaces using MAPAL, the components in an alloy system can be both a pure element (such as 'Al', 'Ti', etc.) or an alloy (such as 'CrFe', 'AlTi0.5', 'CrFeNi', etc.). This enables mapping higher order MPEAs and allows observing the effect of variation in concentration of either a single element or group of elements.

1.2. How to cite

If you use **MAPAL** for your work, please cite the following article:

To be updated

If you use predictions from **hardness** model, cite the following article:

D. Beniwal, P. Singh, S. Gupta, M. J. Kramer, D. D. Johnson, and P. K. Ray, Distilling physical origins of hardness in multiprincipal element alloys directly from ensemble neural network models, npj Comput. Mater. 8, 153 (2022). doi: <https://doi.org/10.1038/s41524-022-00842-3>

If you use predictions from the **phase prediction** model, cite the following article:

D. Beniwal and P. K. Ray, Learning phase selection and assemblages in high-entropy alloys through a stochastic ensemble averaging model, Comput. Mater. Sci. 197, 110647 (2021). doi: <https://doi.org/10.1016/j.commatsci.2021.110647>

2. Installation

The python library is hosted on PyPI repository (<https://pypi.org/project/mapal/>) to enable a quick pip installation in any Python environment.

Run the following command in your terminal to install MAPAL (internet connection required):

```
pip install mapal
```

Python 3 distribution must be installed on your system.

Compatibility tested on: Windows 10, Windows 11, Ubuntu 20.04, Python 3.10.2

Note: It is strongly recommended to use Tensorflow 2.8.0 for using *pretrained_ML_mods* and *map_ML_predictions* packages in MAPAL. Use of newer versions of tensorflow raises legacy issues that interfere with the loading of trained models.

Additional library requirements (automatically installed during the pip installation): *numpy*; *pandas*; *tensorflow*; *matplotlib*; *plotly*; *scikit-image*

3. Quick notes on packages

3.1. MAPAL in-built packages

mapal.element_data

package to access element properties from the in-built element database

mapal.create_alloys

package to create alloys over user-defined compositional spaces including (binary), (ternary) and (ternary with 1 component fixed)

mapal.alloy_features

package to create alloy features based on two methodologies: (a) *Using an operator to mutate elemental properties*, and (b) *Creating an in-built feature*

mapal.miedema_calc

package to use miedema calculator to get chemical and elastic mixing enthalpies based on Miedema's framework

mapal.preTrained_ML_mods

package to use pre-trained machine learning models available in the package to predict properties of alloys

mapal.map_features

package to map alloy features over continuous compositional spaces in binary, ternary and multicomponent systems

mapal.map_ML_predictions

package to map predictions from pre-trained machine learning models over continuous compositional spaces in binary, ternary and multicomponent systems

3.2. Integrating MAPAL with CoSMoR

CoSMoR stands for Compositional-Stimulus and Model-Response framework that enables interpretation of composition-based ML models by providing the exact quantitative contribution of each feature on the total predictions as a function of composition. The CoSMoR research article and code is available online.

Link to CoSMoR article: <https://doi.org/10.1103/PhysRevMaterials.7.043802>

Link to GitHub code: <https://github.com/IDEAsLab-Materials-Informatics/CoSMoR>

CoSMoR is a separate package that can be efficiently integrated with MAPAL codes. While users can do this manually for now, this functionality will be added to MAPAL in a future update.

4. Packages

4.1. *mapal.element_data*

Package to access element data from in-built element database. Two keys (property key and element key) are required to pull out any value from the database.

Package requirements:

None

List of functions:

[*mapal.element_data.load_element_database*](#)

[*mapal.element_data.el_properties_list*](#)

[*mapal.element_data.el_prop_info*](#)

[*mapal.element_data.get_el_property*](#)

mapal.element_data.load_element_database(database_to_use="mapal_EL1")

loads element database and its information file as a dictionary.

Args:

database to use: str (default="mapal_EL1")

element database source directory. Users can customize database directory to load their own database.

Returns:

dict_db_el: dict

dictionary containing element database

dict_db_el_info: dict

dictionary with information about properties included in element database

Examples

```
>>> # Loading element database and associated information file as dictionary
>>> from mapal import element_data
>>> dict_db_el, dict_db_el_info = element_data.load_element_database()
>>> print(f"Element database (element keys): \n{list(dict_db_el.keys())}")
>>> print(f"\nElement database information (property keys):
\n{list(dict_db_el_info.keys())}")
```

Terminal Output

```
Element database (element keys):
```

```
['Li', 'Be', 'B', 'C', 'N', 'Na', 'Mg', 'Al', 'Si', 'P', 'K', 'Ca', 'Sc',
'Ti', 'V', 'Cr', 'Mn', 'Fe', 'Co', 'Ni', 'Cu', 'Zn', 'Ga', 'Ge', 'As', 'Rb',
'Sr', 'Y', 'Zr', 'Nb', 'Mo', 'Tc', 'Ru', 'Rh', 'Pd', 'Ag', 'Cd', 'In', 'Sn',
'Sb', 'Cs', 'Ba', 'La', 'Ce', 'Pr', 'Nd', 'Pm', 'Sm', 'Eu', 'Gd', 'Tb', 'Dy',
'Ho', 'Er', 'Tm', 'Yb', 'Lu', 'Hf', 'Ta', 'W', 'Re', 'Os', 'Ir', 'Pt', 'Au',
'Hg', 'Tl', 'Pb', 'Bi', 'Th', 'U', 'Pu']
```

Element database information (property keys):

```
['el_name', 'at_no', 'el_type', 'at_wt', 'crys_struct', 'phi_workFunc',
'nWS', 'Vm', 'G', 'K', 'Sf_E', 'Hf_vac', 'R_P_ratio', 'r_met', 'r_cov', 'VEC',
'EN_Pauling', 'EN_Allen', 'density_S', 'Tm', 'Coh_E', 'Compressibility',
'VHN', 'E', 'poisson_num', 'n_unpaired', 'spin_mag_moment']
```

mapal.element_data.el_properties_list(database_to_use="mapal_EL1")

generates the list of all 'property keys' available in the in-built element database

Args:

database_to_use: str (default="mapal_EL1")

element database source directory. Users can customize database directory to load their own database.

Returns:

el_prop_keys: list

list of all property keys available in built-in element database

Examples

```
>>> # Accessing list of all property keys available in database
>>> from mapal import element_data
>>> el_prop_list = element_data.el_properties_list()
>>> print(el_prop_list)
```

Terminal Output

```
['el_name', 'at_no', 'el_type', 'at_wt', 'crys_struct', 'phi_workFunc',
'nWS', 'Vm', 'G', 'K', 'Sf_E', 'Hf_vac', 'R_P_ratio', 'r_met', 'r_cov', 'VEC',
'EN_Pauling', 'EN_Allen', 'density_S', 'Tm', 'Coh_E', 'Compressibility',
'VHN', 'E', 'poisson_num', 'n_unpaired', 'spin_mag_moment']
```

mapal.element_data.el_prop_info(prop_key, database_to_use="mapal_EL1")

prints information about an element property key

Args:

prop_key: str

'property key' for which information is needed

database_to_use: str (default="mapal_EL1")

element database source directory. Users can customize database directory to load their own database.

Returns:

prints information on console

Examples

```
>>> # Printing information about any property key
```

```
>>> from mapal import element_data
>>> element_data.el_prop_info("r_cov")
```

Terminal Output

```
---Generated information about 'r_cov' property key---
Property key: r_cov
fullname: Covalent radius
units: Angstrom
source: WebElements, <https://www.webelements.com>
-----
```

```
mapal.element_data.get_el_property(el, prop_key, database_to_use="mapal_EL1")
```

get property of an element from built-in element database

Args:

el: str

'element key' for which property is needed e.g., "Al" or "Fe"

prop_key: str

'property key' to be searched e.g., "EN_Allen" or "VEC"

database_to_use: str (default="mapal_EL1")

element database source directory. Users can customize database directory to load their own database.

Returns:

el_prop: str/int/float

value of property queried from element database

Examples

```
>>> # Accessing element property using element key and property key
>>> Fe_VEC = element_data.get_el_property("Fe", "VEC")
>>> print(Fe_VEC)
```

Terminal Output

```
8
```

Exceptions

```
ValueError("Property key '%s' does not exist in database." % (prop_key))
```

```
ValueError("Element '%s' not in database." % (el))
```

```
ValueError("Element '%s' and property '%s' not in database." % (el, prop_key))
```

4.2. *mapal.create_alloys*

Package to create alloys over user-defined compositional spaces

Package requirements:

numpy, pandas

List of functions:

`mapal.create_alloys.binary`

`mapal.create_alloys.ternary`

`mapal.create_alloys.ternary\_1Cfixed`

`mapal.create_alloys.equiatomic alloys`

`mapal.create_alloys.all possible compositions`

`mapal.create_alloys.binary(A, B, dc=0.01, cAmin=0, cAmax=1)`

generates compositions for a binary system with two components

Args:

A: str

component 'A': an element (e.g. "Al" or "Fe") or a multicomponent system (e.g. "CrFeNi")

B: str

component 'B': an element (e.g. "Al" or "Fe") or a multicomponent system (e.g. "CrFeNi")

dc: float (optional; default = 0.01; range (0,1))

concentration step size (in atomic fraction)

cAmin: float (optional; default = 0.0; range (0,1))

min concentration of component A

cAmax: float (optional; default = 1.0; range (0,1))

max concentration of component A

Returns:

df_alloys: pandas dataframe

dataframe with alloy names and compositions in at.fractions of component A & B

Examples

```
>>> # Create compositions over a binary system
>>> from mapal import create_alloys
>>> A = "Al"
>>> B = "CoCrFeNi"
>>> df_alloys = create_alloys.binary(A, B, dc=0.01, cAmin=0, cAmax=0.4)
>>> print(df_alloys.head(n=5))
```

Terminal Output

```
-----
Creating alloys for (Al)-(CoCrFeNi)... DONE.
-----
```

	alloy_name	xA[A=Al]	xB[B=CoCrFeNi]	\\
alloy_id				
1	Al0.0Co0.25Cr0.25Fe0.25Ni0.25	0.00	1.00	
2	Al0.01Co0.248Cr0.248Fe0.248Ni0.248	0.01	0.99	
3	Al0.02Co0.245Cr0.245Fe0.245Ni0.245	0.02	0.98	
4	Al0.03Co0.242Cr0.242Fe0.242Ni0.242	0.03	0.97	
5	Al0.04Co0.24Cr0.24Fe0.24Ni0.24	0.04	0.96	

	xEl[Al]	xEl[Co]	xEl[Cr]	xEl[Fe]	xEl[Ni]
alloy_id					
1	0.00	0.250	0.250	0.250	0.250
2	0.01	0.248	0.248	0.248	0.248
3	0.02	0.245	0.245	0.245	0.245
4	0.03	0.242	0.242	0.242	0.242
5	0.04	0.240	0.240	0.240	0.240

```
mapal.create_alloys.ternary(A, B, C, dc=0.01)
```

generates compositions for a ternary system with three components

Args:

A: str

component 'A': an element (e.g. "Al") or a multicomponent system (e.g. "CrFe")

B: str

component 'B': an element (e.g. "Ti") or a multicomponent system (e.g. "CoNi")

C: str

component 'C': an element (e.g. "Zr") or a multicomponent system (e.g. "WMo")

dc: float (optional; default = 0.01; range (0,1))

concentration step size (in atomic fraction)

Returns:

df_alloys: pandas dataframe

dataframe with alloy names and compositions in atomic fractions of component A, B & C

Examples

```
>>> # Create compositions over a ternary system
>>> from mapal import create_alloys
>>> A = "Al"
>>> B = "Ti"
>>> C = "CoCrFeNi"
>>> df_alloys = create_alloys.ternary(A, B, C, dc=0.1)
>>> print(df_alloys.head(n=5))
```

Terminal Output

```
-----  
Creating alloys for (Al)-(Ti)-(CoCrFeNi) ternary system... DONE.  
-----
```

	alloy_name	xA[A=Al]	xB[B=Ti]	\\
alloy_id				
1	Al0.0Co0.25Cr0.25Fe0.25Ni0.25Ti0.0	0.0	0.0	
2	Al0.0Co0.225Cr0.225Fe0.225Ni0.225Ti0.1	0.0	0.1	
3	Al0.0Co0.2Cr0.2Fe0.2Ni0.2Ti0.2	0.0	0.2	
4	Al0.0Co0.175Cr0.175Fe0.175Ni0.175Ti0.3	0.0	0.3	
5	Al0.0Co0.15Cr0.15Fe0.15Ni0.15Ti0.4	0.0	0.4	

	xC[C=CoCrFeNi]	xE1[Al]	xE1[Co]	xE1[Cr]	xE1[Fe]	xE1[Ni]	xE1[Ti]
alloy_id							
1	1.0	0.0	0.250	0.250	0.250	0.250	0.0
2	0.9	0.0	0.225	0.225	0.225	0.225	0.1
3	0.8	0.0	0.200	0.200	0.200	0.200	0.2
4	0.7	0.0	0.175	0.175	0.175	0.175	0.3
5	0.6	0.0	0.150	0.150	0.150	0.150	0.4

```
-----
```

`mapal.create_alloys.ternary_1Cfixed(A, B, C, Cfix, dc=0.01, cAmin=0, cAmax="auto")`

generates compositions for a ternary system where concentration of one component is fixed

Args:

A: str

component 'A': an element (e.g. "Al") or a multicomponent system (e.g. "CrFe")

B: str

component 'B': an element (e.g. "Ti") or a multicomponent system (e.g. "CoNi")

C: str

component 'C': an element (e.g. "Zr") or a multicomponent system (e.g. "WMo")

Cfix: float (range: (0,1))

fixed composition value (atomic fraction) of component C

dc: float (optional; default = 0.01; range (0,1))

concentration step size (in atomic fraction)

cAmin: float (optional; default = 0.0; range (0,1))

min concentration of component A

cAmax: float (optional; default = "auto"; range (0,1))

max conc. of component A. The default is "auto" wherein it is set to max possible value i.e. (1-Cfix)

Returns:

df_alloys: pandas dataframe

dataframe with alloy names and compositions in atomic fractions of component A, B & C

Examples

```
>>> # Create compositions over a ternary system where concentration of one  
component is fixed  
>>> from mapal import create_alloys  
>>> A = "Al"
```

```
>>> B = "Ti"
>>> C = "CoCrFeNi"
>>> df_alloys = create_alloys.ternary_1Cfixed(A, B, C, Cfix=0.5, dc=0.1,
cAmin=0, cAmax=0.5)
>>> print(df_alloys.head(n=10))
```

Terminal Output

```
Concentration of (CoCrFeNi) fixed at '0.500'
Creating alloys for (Al)-(Ti)-(CoCrFeNi) ternary system... DONE.
-----
```

	alloy_name	xA[A=Al]	xB[B=Ti]	\ \			
alloy_id							
1	Al0.0Co0.125Cr0.125Fe0.125Ni0.125Ti0.5	0.0	0.5				
2	Al0.1Co0.125Cr0.125Fe0.125Ni0.125Ti0.4	0.1	0.4				
3	Al0.2Co0.125Cr0.125Fe0.125Ni0.125Ti0.3	0.2	0.3				
4	Al0.3Co0.125Cr0.125Fe0.125Ni0.125Ti0.2	0.3	0.2				
5	Al0.4Co0.125Cr0.125Fe0.125Ni0.125Ti0.1	0.4	0.1				
6	Al0.5Co0.125Cr0.125Fe0.125Ni0.125Ti0.0	0.5	0.0				

	xC[C=CoCrFeNi]	xE1[Al]	xE1[Co]	xE1[Cr]	xE1[Fe]	xE1[Ni]	xE1[Ti]
alloy_id							
1	0.5	0.0	0.125	0.125	0.125	0.125	0.5
2	0.5	0.1	0.125	0.125	0.125	0.125	0.4
3	0.5	0.2	0.125	0.125	0.125	0.125	0.3
4	0.5	0.3	0.125	0.125	0.125	0.125	0.2
5	0.5	0.4	0.125	0.125	0.125	0.125	0.1
6	0.5	0.5	0.125	0.125	0.125	0.125	0.0

mapal.create_alloys.equiatomic_alloys(el_list, n_el)

generates all possible equiatomic compositions from a set of elements

Args:

el_list: list

list of all elements to be included

n_el: int

number of elements in alloys

Returns:

df_alloy_names: pandas dataframe

dataframe with all possible equiatomic alloy names

Examples

```
>>> # Create all possible equiatomic alloys from a list of elements
>>> from mapal import create_alloys
>>> el_list = ["Al", "Co", "Ni", "Fe", "Cr", "Ti", "Nb"]
>>> n_el = 5
>>> df_alloys = create_alloys.equiatomic_alloys(el_list, n_el)
>>> print("Number of alloys created: ", len(df_alloys))
>>> print(df_alloys.head(5))
```

Terminal Output

```
Creating all equiatomic alloys with 5 elements from ['Al', 'Co', 'Ni', 'Fe', 'Cr', 'Ti', 'Nb'] ... DONE.
```

```
-----
Number of alloys created: 21
```

```
    alloy_name
0  AlCoNiFeCr
1  AlCoNiFeTi
2  AlCoNiFeNb
3  AlCoNiCrTi
4  AlCoNiCrNb
```

```
-----
mapal.create_alloys.all_possible_compositions(el_list, dc=0.01, cmin=0.0)
```

generates all possible compositions from given list of elements

Args:

el_list: list
list of all elements to be included
dc: float (optional; default = 0.01; range (0,1))
concentration step size (in atomic fraction)
cmin: float (optional; default = 0.0; range (0,1))
minimum concentration of each element

Returns:

df_alloys: pandas dataframe
dataframe with alloy names and compositions in atomic fractions for all possible alloys

Examples

```
>>> # Create all possible alloys from a list of elements
>>> from mapal import create_alloys
>>> el_list = ["Al", "Co", "Cr", "Fe", "Ni"]
>>> df_alloys = create_alloys.all_possible_compositions(el_list, dc=0.05,
cmin=0.05)
>>> print("Number of alloys created: ", len(df_alloys))
>>> print(df_alloys)
```

Terminal Output

```
-----
Creating all possible alloys containing ['Al', 'Co', 'Cr', 'Fe', 'Ni']
elements DONE.
```

```
-----
Number of alloys created: 3876
```

```
    alloy_name  xEl[Al]  xEl[Co]  xEl[Cr]  xEl[Fe]
\\
alloy_id
1      Al0.05Co0.05Cr0.05Fe0.05Ni0.8      0.05      0.05      0.05      0.05
2      Al0.05Co0.05Cr0.05Fe0.1Ni0.75      0.05      0.05      0.05      0.10
3      Al0.05Co0.05Cr0.05Fe0.15Ni0.7      0.05      0.05      0.05      0.15
4      Al0.05Co0.05Cr0.05Fe0.2Ni0.65      0.05      0.05      0.05      0.20
5      Al0.05Co0.05Cr0.05Fe0.25Ni0.6      0.05      0.05      0.05      0.25
...
3872    Al0.75Co0.05Cr0.05Fe0.05Ni0.1      0.75      0.05      0.05      0.05
```

3873	Al0.75Co0.05Cr0.05Fe0.1Ni0.05	0.75	0.05	0.05	0.10
3874	Al0.75Co0.05Cr0.1Fe0.05Ni0.05	0.75	0.05	0.10	0.05
3875	Al0.75Co0.1Cr0.05Fe0.05Ni0.05	0.75	0.10	0.05	0.05
3876	Al0.8Co0.05Cr0.05Fe0.05Ni0.05	0.80	0.05	0.05	0.05

	xEl[Ni]
alloy_id	
1	0.80
2	0.75
3	0.70
4	0.65
5	0.60
...	...
3872	0.10
3873	0.05
3874	0.05
3875	0.05
3876	0.05

4.3. *mapal.alloy_features*

Package to create various alloy features

Package requirements:

numpy

List of functions:

mapal.alloy_features.[list_operators](#)

mapal.alloy_features.[list_features](#)

mapal.alloy_features.[get_feat_value](#)

mapal.alloy_features.[get_comp_dict](#)

mapal.alloy_features.[get_comp_avg](#)

mapal.alloy_features.[get_asymmetry](#)

mapal.alloy_features.[get_local_mismatch](#)

mapal.alloy_features.[get_sqdiff](#)

mapal.alloy_features.[get_modulus_mismatch](#)

mapal.alloy_features.[get_S_config](#)

mapal.alloy_features.[get_Singh_parameter](#)

mapal.alloy_features.[get_Wang_parameter](#)

mapal.alloy_features.[get_latt_dist_energy](#)

mapal.alloy_features.[get_energy_strength_model](#)

mapal.alloy_features.[get_Peierl_Nabarro_factor](#)

`mapal.alloy_features.list_operators()`

return list of operators available to mutate elemental properties into alloy features

Args:

no inputs required

Returns:

op_list: list

list of operators available

Examples

```
>>> # print list of operator keys available to mutate elemental properties
to alloy features
>>> from mapal import alloy_features
>>> alloy_features.list_operators()
```

Terminal Output

```
['comp_avg', 'asymmetry', 'local_mismatch', 'sqdiff', 'modulus_mismatch']
```

`mapal.alloy_features.list_features()`

return list of alloy features available

Args:

no inputs required

Returns:

feat_list: list

list of features available

Examples

```
>>> # print list of alloy features available along with their respective
function calls
>>> from mapal import alloy_features
>>> alloy_features.list_features()
```

Terminal Output

```
['S_config', 'Singh_parameter', 'Wang_parameter', 'latt_dist_energy',
'energy_strength_model', 'Peierl_Nabarro_factor']
```

`mapal.alloy_features.get_feat_value(alloy_list, feature)`

serves as a single access function to calculate all the features available in MAPAL. It returns feature values for a single alloy or a list of alloys.

Args:

alloy_list: str/list/array

list of alloy names

feature: str/tuple

feature to be calculated, can be either a feature string (e.g. “S_config”) or (operator, el_prop) tuple e.g. (“comp_avg”, “VEC”)

Returns:

feat_val_list: list
list of feature values

Examples

```
>>> import MAPAL
>>> from mapal import alloy_features

>>> alloy_names_list = ["CoCrFeNi", "Al0.25CoCrFeNi", "Al0.5CoCrFeNi",
                        "Al0.75CoCrFeNi", "AlCoCrFeNi"]

>>> # Operator and element property can be combined as a tuple to form a
feature
>>> feat1 = ("comp_avg", "VEC")
>>> feat1_val_list = alloy_features.get_feat_value(alloy_names_list, feat1)
>>> print(feat1, " values: ", feat1_val_list)

>>> # A feature can be selected from the list of features
>>> feat2 = "S_config"
>>> feat2_val_list = alloy_features.get_feat_value(alloy_names_list, feat2 )
>>> print(feat2, " values: ", feat2_val_list )
```

Terminal Output

```
Calculating 'comp_avg_VEC' feature values... DONE.
('comp_avg', 'VEC') values: [7.25, 7.9411764705882355, 7.666666666666667,
7.421052631578947, 7.200000000000001]
Calculating 'S_config' feature values... DONE.
S_config values: [0.01152565131835077, 0.012707663913447362,
0.013145213438928542, 0.01333206209759665, 0.01338086680397711]
```

mapal.alloy_features.get_comp_dict(alloy_name)

creates 'dictionary' with alloy composition from alloy name string

Args:

alloy_name: str
alloy name string e.g. "NiAl2", "BaTiO3", "Al1.5Ti0.25CrFeNi"

Returns:

dict_alloy_comp: dict
composition dict with two keys:
"el_list": list of elements in alloys
"el_at_frac_list": list of atomic fraction of each element in el_list

Examples

```
>>> # Create composition of an alloy from alloy name
>>> from mapal import alloy_features
>>> alloy_name = "Al1.5Ti0.5CrFeNi"
>>> dict_comp = alloy_features.get_comp_dict(alloy_name)
>>> print(dict_comp)
```

Terminal Output

```
{'el_list': ['Al', 'Cr', 'Fe', 'Ni', 'Ti'],
'el_at_frac_list': [0.3, 0.2, 0.2, 0.2, 0.1]}
```

mapal.alloy_features.get_comp_avg(alloy_name, feat_key)

calculates composition-weighted average value ($\bar{X}$) of any feature (X) for an alloy.

Mathematical expression: $\bar{X} = \sum_{i=1}^n c_i X_i$

where c_i is the atomic fraction and X_i is the feature value of i^{th} element

Args:

alloy_name: str/list/array

alloy name string e.g. "NiAl2", "BaTiO3", "Al1.5Ti0.25CrFeNi"

feat_key: str

feature key e.g. "r_cov" or "EN_Allen". This can be any feature key available in built-in element database (The list can be accessed using mapal.element_data.el_properties)

Returns:

avg_feat: float/list

composition-weighted average value of feature

Examples

```
>>> # Calculate composition-weighted average feature for an alloy
>>> from mapal import alloy_features
>>> alloy_name = "Al1.5Ti0.5CrFeNi"
>>> feat_key = "r_cov"
>>> c_avg_rcov = alloy_features.get_comp_avg(alloy_name, feat_key)
>>> print(c_avg_rcov)
```

Terminal Output

```
1.2619999999999998
```

mapal.alloy_features.get_asymmetry(alloy_name, feat_key)

calculates composition-weighted asymmetry value (δ_X) of any feature (X) for an alloy

Mathematical expression: $\delta_X = \sqrt{\sum_{i=1}^n c_i (1 - (X_i/\bar{X}))^2}$

where: c_i is the atomic fraction and X_i is the feature value of i^{th} element; $\bar{X}$ is the composition-weighted average of feature X

Args:

alloy_name: str/list/array

alloy name string e.g. "NiAl2", "BaTiO3", "Al1.5Ti0.25CrFeNi"

feat_key: str

feature key e.g. "r_cov" or "EN_Allen". This can be any feature key available in built-in element database (The list can be accessed using mapal.element_data.el_properties)

Returns:

asymm_feat: float/list

composition-weighted asymmetry value of feature

Examples

```
>>> # Calculate composition-weighted asymmetry feature for an alloy
>>> from mapal import alloy_features
>>> alloy_name = "Al1.5Ti0.5CrFeNi"
>>> feat_key = "r_cov"
>>> c_asymm_rcov = alloy_features.get_asymmetry(alloy_name, feat_key)
>>> print(c_asymm_rcov)
```

Terminal Output

```
0.06618260861433
```

`mapal.alloy_features.get_local_mismatch(alloy_name, feat_key)`

calculates local-mismatch value (LM_X) of any feature (X) for an alloy

Mathematical expression: $LM_X = \sum_{i=1}^n \sum_{j=1, j \neq i}^n c_i c_j |X_i - X_j|$

where: c_i is the atomic fraction and X_i is the feature value of i^{th} element

Args:

alloy_name: str/list/array

alloy name string e.g. "NiAl2", "BaTiO3", "Al1.5Ti0.25CrFeNi"

feat_key: str

feature key e.g. "r_cov" or "EN_Allen". This can be any feature key available in built-in element database (The list can be accessed using `mapal.element_data.el_properties`)

Returns:

loc_mismatch_feat: float/list

local-mismatch value of feature for alloy

Examples

```
>>> # Calculate local-mismatch value of a feature for an alloy
>>> from mapal import alloy_features
>>> alloy_name = "Al1.5Ti0.5CrFeNi"
>>> feat_key = "r_cov"
>>> loc_mismatch_rcov = alloy_features.get_local_mismatch(alloy_name,
>>> feat_key)
>>> print(loc_mismatch_rcov)
```

Terminal Output

```
0.040000000000000002
```

`mapal.alloy_features.get_sqdiff(alloy_name, feat_key)`

calculates composition-weighted average of square-difference value ($\overline{SD}_X$) of any feature (X) for an alloy

Mathematical expression: $\overline{SD}_X = \sqrt{\sum_{i=1}^n c_i (X_i - \bar{X})^2}$

where: c_i is the atomic fraction and X_i is the feature value of i^{th} element; $\bar{X}$ is the composition-weighted average of feature X

Args:

alloy_name: str/list/array

alloy name string e.g. "NiAl2", "BaTiO3", "Al1.5Ti0.25CrFeNi"

feat_key: str

feature key e.g. "r_cov" or "EN_Allen". This can be any feature key available in built-in element database (The list can be accessed using `mapal.element_data.el_properties`)

Returns:

sqDiff_feat: float/list

composition-weighted average of square-difference value of any feature for an alloy

Examples

```
>>> # Calculate composition-weighted average of square-difference value of
any feature for an alloy
>>> from mapal import alloy_features
>>> alloy_name = "Al1.5Ti0.5CrFeNi"
>>> feat_key = "r_cov"
>>> sqDiff_rcov = alloy_features.get_sqdiff(alloy_name, feat_key)
>>> print(sqDiff_rcov)
```

Terminal Output

```
0.08352245207128442
```

`mapal.alloy_features.get_modulus_mismatch(alloy_name, modulus_key)`

calculates modulus mismatch parameter (η) for an alloy

Mathematical expression: $\eta_X = \sum_{i=1}^N \left(\frac{c_i \frac{2(X_i - \bar{X})}{X_i + \bar{X}}}{1 + 0.5 \left(c_i \frac{2(X_i - \bar{X})}{X_i + \bar{X}} \right)} \right)$

where: c_i is the atomic fraction and X_i is the modulus of i^{th} element; $\bar{X}$ is the composition-weighted average of modulus X for alloy; X can be Elastic (E), Shear (G) or Bulk modulus (K)

Args:

alloy_name: str/list/array

alloy name string e.g. "NiAl2", "BaTiO3", "Al1.5Ti0.25CrFeNi"

modulus_key: str

modulus key: "E"-Youngs modulus, "G"-Shear modulus and "K"-Bulk modulus

Returns:

mod_mismatch: float/list

modulus mismatch parameter for an alloy

Examples

```
>>> # Calculate modulus mismatch parameter for an alloy
>>> from mapal import alloy_features
>>> alloy_name = "Al1.5Ti0.5CrFeNi"
>>> modulus_key = ["E", "G", "K"]
```

```
>>> mod_mismatch = [alloy_features.get_modulus_mismatch(alloy_name, mod) for
mod in modulus_key]
>>> print(mod_mismatch)
```

Terminal Output

```
[-0.08130175136122864, -0.11120523570064911, -0.059877262184347115]
```

mapal.alloy_features.get_S_config(alloy_name)

calculates configurational entropy of mixing (ΔS^{config}) for an alloy

Mathematical expression: $\Delta S^{\text{config}} = -R \sum_{i=1}^N c_i \log(c_i)$

where: c_i is the atomic fraction of i^{th} element; R is the universal gas constant (=8.314 J/K.mol)

Args:

alloy_name: str/list/array

alloy name string e.g. "NiAl2", "BaTiO3", "Al1.5Ti0.25CrFeNi"

Returns:

S_config: float/list

configurational entropy of mixing for alloy (units: kJ/K.mol)

Examples

```
>>> # Calculate configurational entropy of mixing for an alloy
>>> from mapal import alloy_features
>>> alloy_name = "Al1.5Ti0.5CrFeNi"
>>> S_config = alloy_features.get_S_config(alloy_name)
>>> print(S_config)
```

Terminal Output

```
0.012945838297251266
```

mapal.alloy_features.get_Singh_parameter(alloy_name)

calculates geometrical parameter (Λ) proposed by Singh et al. for multicomponent alloys[1]

Mathematical expression: $\Lambda = \frac{\Delta S^{\text{config}}}{(\delta_r)^2}$

where: ΔS^{config} is the configurational entropy of alloy and δ_r is the composition-weighted atomic size asymmetry of alloy

Args:

alloy_name: str/list/array

alloy name string e.g. "NiAl2", "BaTiO3", "Al1.5Ti0.25CrFeNi"

Returns:

Singh_param: float/list

value of Singh's geometrical parameter

Examples

```
>>> # Calculate Singh's geometrical parameter for an alloy
>>> from mapal import alloy_features
```

```
>>> alloy_name = "Al1.5Ti0.5CrFeNi"
>>> Singh_param = alloy_features.get_Singh_parameter(alloy_name)
>>> print(Singh_param)
```

Terminal Output

```
2.568810361749183
```

`mapal.alloy_features.get_Wang_parameter(alloy_name)`

calculates Wang's geometrical solid-angle ratio parameter (γ) proposed by Wang et al. for multicomponent alloys [2]

Mathematical expression: $\gamma = \omega_S / \omega_L$

where: $\omega_S = 1 - \sqrt{1 - \left(\frac{\bar{r}}{r_S + \bar{r}}\right)^2}$; $\omega_L = 1 - \sqrt{1 - \left(\frac{\bar{r}}{r_L + \bar{r}}\right)^2}$

ω_S and ω_L are the solid angles around smallest and largest atoms

r_S and r_L are the radii of smallest and largest atoms in alloy

$\bar{r}$ is the composition-weighted average radius of alloy

Args:

alloy_name: str/list/array

alloy name string e.g. "NiAl2", "BaTiO3", "Al1.5Ti0.25CrFeNi"

Returns:

wang_param: float/list

value of Wang's geometrical solid-angle ratio parameter

Examples

```
>>> # Calculate Wang's geometrical solid-angle ratio parameter
>>> from mapal import alloy_features
>>> alloy_name = "Al1.5Ti0.5CrFeNi"
>>> wang_param = alloy_features.get_Wang_parameter(alloy_name)
>>> print(wang_param)
```

Terminal Output

```
1.1938422815478549
```

`mapal.alloy_features.get_latt_dist_energy(alloy_name)`

calculates lattice distortion energy parameter (μ) for an alloy

Mathematical expression: $\mu = 0.5 \bar{E} \delta_r$

where: $\bar{E}$ is the composition-weighted average of elastic modulus for alloy; δ_r is the composition-weighted atomic size asymmetry of alloy

Args:

alloy_name: str/list/array

alloy name string e.g. "NiAl2", "BaTiO3", "Al1.5Ti0.25CrFeNi"

Returns:

latt_dist_energy: float/list

lattice distortion energy parameter

Examples

```
>>> # Calculate lattice distortion energy parameter for an alloy
>>> from mapal import alloy_features
>>> alloy_name = "Al1.5Ti0.5CrFeNi"
>>> latt_dist_energy = alloy_features.get_latt_dist_energy(alloy_name)
>>> print(latt_dist_energy)
```

Terminal Output

```
5.1986203318598285
```

mapal.alloy_features.get_energy_strength_model(alloy_name)

calculates energy term (A) used in strengthening model

Mathematical expression: $A = \bar{G} \delta_r \left(\frac{1+\bar{\mu}}{1-\bar{\mu}} \right)$

where: $\bar{G}$ is the composition-weighted average of shear modulus for alloy; δ_r is the composition-weighted atomic size asymmetry of alloy; $\bar{\mu}$ is the composition-weighted average of Poisson's number for alloy

Args:

alloy_name: str/list/array

alloy name string e.g. "NiAl2", "BaTiO3", "Al1.5Ti0.25CrFeNi"

Returns:

energy_term: float/list

energy term value

Examples

```
>>> # Calculate energy term used in strengthening model
>>> from mapal import alloy_features
>>> alloy_name = "Al1.5Ti0.5CrFeNi"
>>> energy_term = alloy_features.get_energy_strength_model(alloy_name)
>>> print(energy_term)
```

Terminal Output

```
9.152562154416206
```

mapal.alloy_features.get_Peierl_Nabarro_factor(alloy_name)

calculates Peierls-Nabarro factor

Mathematical expression: $F = \frac{2\bar{G}}{1-\bar{\mu}}$

where: $\bar{G}$ and $\bar{\mu}$ is the composition-weighted average of shear modulus and Poisson's number for alloy, respectively

Args:

alloy_name: str/list/array

alloy name string e.g. "NiAl2", "BaTiO3", "Al1.5Ti0.25CrFeNi"

Returns:

PN_factor: float/list
Peierls-Nabarro factor

Examples

```
>>> # Calculate Peierls-Nabarro factor for an alloy
>>> from mapal import alloy_features
>>> alloy_name = "Al1.5Ti0.5CrFeNi"
>>> PN_factor = alloy_features.get_Peierl_Nabarro_factor(alloy_name)
>>> print(PN_factor)
```

Terminal Output

```
195.49192364170335
```

References:

- [1]. Singh et al., A geometrical parameter for the formation of disordered solid solutions in multi-component alloys, Intermetallics, Vol 53, 2014, doi: <https://doi.org/10.1016/j.intermet.2014.04.019>
- [2]. Wang et al., Atomic-size effect and solid solubility of multicomponent alloys, Scripta Materialia, Vol 94, 2015, doi: <https://doi.org/10.1016/j.scriptamat.2014.09.010>
-

4.4. *mapal.miedema_calc*

Package to implement Miedema's theory of alloy cohesion to estimate mixing enthalpies

Package requirements:

numpy

List of functions:

`mapal.miedema_calc.get_miedema_enthalpy`

```
mapal.miedema_calc.get_miedema_enthalpy(alloy_name, state="liquid", show_calc=False,
model="R")
```

Calculates binary enthalpies using Miedema's model [1,2] and extends it to multi-component alloys using Takeuchi and Inoue's extended regular solid solution [3].

Args:

alloy_name: str

alloy name string e.g. "NiAl2", "BaTiO3", "Al1.5Ti0.25CrFeNi"

state: str (optional, default= "liquid")

state of alloy; possible values: "liquid" or "solid"; this affects calculation only when the binary pair has a transition and non-transition

show_calc: bool (optional, default=False)

if set to 'True', prints all calculation steps on the terminal

model: str (optional, default="R")

"R"-regular model: Equiatomic binary enthalpies are extended to multi-component alloy

"NR"-non regular model: Original concentrations are used straight-away for enthalpy calculations

Returns:

enthalpy_dict: dict

enthalpy dict with five keys:

"H_chem": chemical enthalpy of mixing

"H_el": elastic enthalpy of mixing

"H_mix": total enthalpy of mixing (=H_chem+H_el)

"H_IM": formation enthalpy of intermetallic

"units": units of enthalpy (kJ/mol)

Examples

```
>>> # Using Miedema calculator for enthalpy calculation
```

```
>>> from mapal import miedema_calc
>>> alloy_name = "Al1.5Ti0.5CrFeNi"
>>> H_dict = miedema_calc.get_miedema_enthalpy(alloy_name)
>>> print(H_dict)
```

Terminal Output

```
{'H_chem': -20.2406,
 'H_el': 6.428,
 'H_mix': -13.8126,
 'H_IM': -28.3401,
 'units': 'kJ/mol'}
```

References

- [1]. Miedema et al., Cohesion in alloys — fundamentals of a semi-empirical model, Physica B+C, Vol 100, 1980, doi: [https://doi.org/10.1016/0378-4363\(80\)90054-6](https://doi.org/10.1016/0378-4363(80)90054-6)
 - [2]. Zhang et al., Miedema Calculator: A thermodynamic platform for predicting formation enthalpies of alloys within framework of Miedema's Theory, Computer Physics Communications, Vol 209, 2016, doi: <https://doi.org/10.1016/j.cpc.2016.08.013>
 - [3]. Takeuchi & Inoue, Calculations of Mixing Enthalpy and Mismatch Entropy for Ternary Amorphous Alloys, Materials Transactions, JIM, Vol 41, 2000, doi: <https://doi.org/10.2320/matertrans1989.41.1372>
 - [4]. H. Bakker, Enthalpies in alloys — Miedema's Semi-Empirical Model, Trans Tech Publications, 1998.
-

4.5. *mapal.preTrained_ML_mods*

Package to use pre-trained machine learning (ML) models for making predictions for alloy systems

Package requirements:

numpy, pandas, tensorflow==2.8.0

List of functions:

`mapal.preTrained_ML_mods.mods\_list`

`mapal.preTrained_ML_mods.get\_mod\_info`

`mapal.preTrained_ML_mods.predict`

`mapal.preTrained_ML_mods.mods_list()`

prints models available along with model keys to access a model

Args:

no inputs required

Returns:

prints quick information about pre-trained ML models available

Examples

```
>>> # print the pre-trained machine learning models available
>>> from mapal import preTrained_ML_mods;
>>> preTrained_ML_mods.mods_list();
```

Terminal Output

```
-----
--- Pre-trained ML models available in MAPAL ---
model_key-'M1': Predicts vickers hardness for as-cast multi-principal element
alloys
model_key-'M2': Predicts phase fractions of FCC, BCC & IM phases for multi-
principal element alloys
-----
```

`mapal.preTrained_ML_mods.get_mod_info(mod_key="all")`

generates detailed information about any model key

Args:

mod_key: str (default="all")

model key for which information is required e.g. "M1" or "M2".

Returns:

dict_info: dict

dictionary with information about queried model; it has 7 keys:

"model_key": model key used for access

"description": brief description of model

"reference": Reference of the model (usually a published work)

"DOI": digital object identifier of model reference

"features": list of features used by the model

"model_output": the output produced by the model

"model_input": inputs taken by the model

Examples

```
>>> # print the information about a pre-trained machine learning model
>>> from mapal import preTrained_ML_mods;
>>> mod_key = "M2";
>>> M2_info = preTrained_ML_mods.get_mod_info(mod_key);
>>> print(M2_info);
```

Terminal Output

```
{'model_key': 'M2',
 'description': 'Predicts phase fractions of FCC, BCC & IM phases for multi-
principal element alloys',
 'reference': 'D Beniwal and PK Ray, Learning phase selection and assemblages
in High-Entropy Alloys through a stochastic ensemble-averaging model,
Computational Materials Science 197 (2021) 110647',
 'DOI': '<https://doi.org/10.1016/j.commatsci.2021.110647>',
 'features': ['R_delta', 'S_config', 'VEC', 'R_cov_delta', 'density_avg',
'E_delta', 'G_delta', 'K_delta', 'EN_Allen_avg', 'Vm_delta', 'E_coh_avg',
'E_coh_delta', 'H_ch_M_L_R', 'H_el_M_S_R'],
 'model_output': 'Three outputs (Phase fractions of FCC, BCC and Intermetallic
phases)',
 'model_input': 'Takes alloy name as input -> converts it to a set of 14
physical & thermodynamic features that act as input for ML model'}
```

```
mapal.preTrained_ML_mods.predict(mod_key, input_alloys, input_contains_concentration =
False)
```

generate predictions using a pre-trained model accessed through "mod_key"

Args:

mod_key: str

model key that identifies each pre-trained model. Use

"preTrained_ML_mods.mods_list()" function to see all models available

input_alloys: list/array/pandas DataFrame

contains input alloys for which prediction is to be made

input_contains_concentration: boolean (default=False)

boolean to decide if the input_alloys contains concentration of elements or not. If set to False, then the input_alloys is first converted to a dataframe with concentration information. The alloys created using mapal.create_alloys contain element concentrations.

Returns:

ML_prediction: array/dictionary/pandas dataframe

Output predicted by the model accessed. The data type of output may vary from model to model. Use "preTrained_ML_mods.get_mod_info(mod_key)" function to see information of each model.

Example 1

```
>>> # Generate predictions for a LIST OF ALLOY NAMES
>>> from mapal import preTrained_ML_mods;
>>> alloy_names_list = ["CoCrFeNi", "Al0.25CoCrFeNi", "Al0.5CoCrFeNi",
"Al0.75CoCrFeNi", "AlCoCrFeNi"];
>>>
>>> # 'input_contains_concentration' will be set to False as the alloy list
has only alloy names
>>> HV_predicted = preTrained_ML_mods.predict("M1", alloy_names_list,
input_contains_concentration=False)
>>> print(HV_predicted);

>>> dict_phase_predict= preTrained_ML_mods.predict("M2", alloy_names_list,
input_contains_concentration=False)
>>> print(dict_phase_predict);
```

Terminal Output

```
-----
--- Pre-trained model loaded (M1: Vickers hardness prediction) ---
Creating alloy features... DONE.
Normalizing features... DONE.
Predicting vickers hardness... DONE.
-----
[150.12535627 159.13735523 293.80784884 466.50351285 512.45991322]

-----
--- Pre-trained model loaded (M2: FCC, BCC & Intermetallic phase fraction
prediction) ---
Creating alloy features... DONE.
Normalizing features... DONE.
Predicting phase fraction of FCC, BCC & Intermetallic phases... DONE.
-----
{'f_FCC': array([0.99812519, 0.98395188, 0.59580822, 0.36225888,
0.01587714]),
 'f_BCC': array([2.80918441e-04, 1.64855047e-03, 3.99954186e-01, 6.33994925e-
01, 9.80873367e-01]),
 'f_IM': array([0.00159389, 0.01439957, 0.00423759, 0.00374619, 0.0032495 ])}
```

Example 2

```
>>> # Generate predictions for ALLOYS CREATED USING "mapal.create_alloys"
>>> from mapal import preTrained_ML_mods;
>>> A, B = "Al", "CoCrFeNi"
>>> df_alloys = create_alloys.binary(A, B, dc=0.01, cAmin=0, cAmax=0.4)
>>>
```

```
>>> # 'input_contains_concentration' will be set to True as df_alloys has
element concentrations
>>> HV_predicted = preTrained_ML_mods.predict("M1", df_alloys,
input_contains_concentration=True)
>>> print(HV_predicted);

>>> dict_phase_predict= preTrained_ML_mods.predict("M2", df_alloys,
input_contains_concentration=True)
>>> print(dict_phase_predict);
```

Terminal Output

```
-----
--- Pre-trained model loaded (M1: Vickers hardness prediction) ---
Creating alloy features... DONE.
Normalizing features... DONE.
Predicting vickers hardness... DONE.
-----
[150.12535627 150.94089883 156.13788993 158.18236042 157.84957086
 157.460632 159.87909389 172.56749545 187.11878473 206.78791791
 242.22097048 293.09657907 368.54469558 427.26249242 457.26035545
 464.22648389 468.32598304 483.39925408 510.84675903 512.968959
 512.45991322 512.19971942 514.35867901 511.75203598 507.09287461
 502.3009122 498.52378706 496.55751787 497.25677453 498.32007224
 498.91104422 495.27408965 492.14764293 490.66050822 490.61785667
 490.9738039 491.39655262 492.30544711 495.24092777 498.13471865
 500.18097997]

-----
--- Pre-trained model loaded (M2: FCC, BCC & Intermetallic phase fraction
prediction) ---
Creating alloy features... DONE.
Normalizing features... DONE.
Predicting phase fraction of FCC, BCC & Intermetallic phases... DONE.
-----
{'f_FCC': array([9.98125194e-01, 9.93703195e-01, 9.88243105e-01, 9.83805433e-
01, 9.74977144e-01, 9.78674607e-01, 9.84053772e-01, 9.88092301e-01,
 9.85859580e-01, 9.68513732e-01, 8.06287877e-01, 5.97406244e-01,
 5.48247168e-01, 5.07653778e-01, 4.49034908e-01, 4.00392296e-01,
 3.48126134e-01, 2.06769526e-01, 6.02924181e-02, 2.37931045e-02,
 1.58771386e-02, 1.10424761e-02, 8.49272895e-03, 6.87163775e-03,
 5.37370480e-03, 4.37174448e-03, 3.53595386e-03, 2.91273431e-03,
 2.34374590e-03, 1.93351655e-03, 1.58100649e-03, 1.28321567e-03,
 1.07826505e-03, 9.24219852e-04, 7.74561446e-04, 6.45586453e-04,
 5.49934164e-04, 4.71674965e-04, 3.95571907e-04, 3.41229286e-04,
 3.03745964e-04]),
 'f_BCC': array([2.80918441e-04, 2.87915258e-04, 3.55751450e-04, 4.85511898e-
04, 6.66545214e-04, 9.73847934e-04, 1.78632546e-03, 3.87854194e-03,
 7.49607375e-03, 2.56607995e-02, 1.89050005e-01, 3.98429333e-01,
 4.47571242e-01, 4.88178369e-01, 5.46948840e-01, 5.95761810e-01,
 6.48152536e-01, 7.89639532e-01, 9.36272352e-01, 9.72858686e-01,
 9.80873367e-01, 9.85756519e-01, 9.88178539e-01, 9.89545340e-01,
 9.91091178e-01, 9.92108382e-01, 9.92935580e-01, 9.93814304e-01,
 9.94538294e-01, 9.95013498e-01, 9.95454852e-01, 9.95700525e-01,
 9.95901252e-01, 9.95986394e-01, 9.96061219e-01, 9.96034590e-01,
 9.96086587e-01, 9.96086396e-01, 9.95981966e-01, 9.95651416e-01,
 9.95396217e-01])}
```

```
'f_IM': array([0.00159389, 0.00600889, 0.01140115, 0.01570906, 0.02435631,
0.02035154, 0.01415991, 0.00802915, 0.00664435, 0.00582547,
0.00466212, 0.00416442, 0.00418159, 0.00416785, 0.00401625,
0.00384589, 0.00372133, 0.00359094, 0.00343522, 0.00334821,
0.0032495 , 0.00320101, 0.00332872, 0.00358302, 0.00353512,
0.00351987, 0.00352846, 0.00327296, 0.00311796, 0.00305299,
0.00296414, 0.00301626, 0.00302048, 0.00308938, 0.00316422,
0.00331982, 0.00336348, 0.00344193, 0.00362247, 0.00400735,
0.00430004]) }
```

4.5.1 References for pre-trained ML models included in MAPAL:

M1-Predicts vickers hardness for as-cast multi-principal element alloys

"D Beniwal, P Singh, S Gupta, MJ Kramer, DD Johnson, PK Ray, Distilling physical origins of hardness in multi-principal element alloys directly from ensemble neural network models, npj Computational Materials, 8 (2022) 153.", "DOI": "<https://doi.org/10.1038/s41524-022-00842-3>"

M2-Predicts phase fractions of FCC, BCC & IM phases for as-cast multi-principal element alloys

"D Beniwal and PK Ray, Learning phase selection and assemblages in High-Entropy Alloys through a stochastic ensemble-averaging model, Computational Materials Science 197 (2021) 110647", "DOI": "<https://doi.org/10.1016/j.commatsci.2021.110647>"

4.6. *mapal.map_features*

Package to map alloy features over continuous compositional spaces in binary, ternary and multicomponent systems

Package requirements:

numpy, pandas, tqdm, matplotlib, plotly, scikit-image

List of functions:

`mapal.map_features.binary`

`mapal.map_features.ternary`

`mapal.map_features.ternary1Cfixed`

`mapal.map_features.binary(feats, A, B, dc=0.01, cAmin=0, cAmax=1, nCols=4)`

maps features to composition variations in binary system

Args:

feats: list

list of features to be mapped

A: str

component A e.g. "Al" or "Al0.5Ti"

B: str

component B e.g. "Al" or "Al0.5Ti"

dc: float (optional; default = 0.01; range (0,1))

concentration step size (in atomic fraction)

cAmin: float (optional, default=0.0)

min concentration of component A (in atomic fraction)

cAmax: float (optional; default = 1.0; range (0,1))

max concentration of component A (in atomic fraction)

nCols: int

max no. of columns in final figure

Returns:

fig: matplotlib figure object

figure containing plots of all features

df_alloys: pandas dataframe

dataframe with all alloys and corresponding feature values

Examples

```
>>> # Mapping features to composition variations in binary system
>>> from mapal import map_features;
>>> feats = ["S_config", ("comp_avg", "VEC"), ("asymmetry", "r_met")];
>>> A, B = "Al", "CrFeNi";
>>> fig, df_alloy = map_features.binary(feats, A, B, dc=0.01, cAmin=0,
cAmax=1, nCols=4);
>>> print(df_alloy );

>>> # Saving the figure image
>>> fig_savename = "(%s)x-(%s)-binaryFeats.png" % (A, B);
>>> fig.savefig(fig_savename, facecolor="white", edgecolor='none');

>>> # Saving the data
>>> df_savename = "(%s)x-(%s)-binaryFeats.csv" % (A, B);
>>> df_alloy.to_csv(df_savename);
```

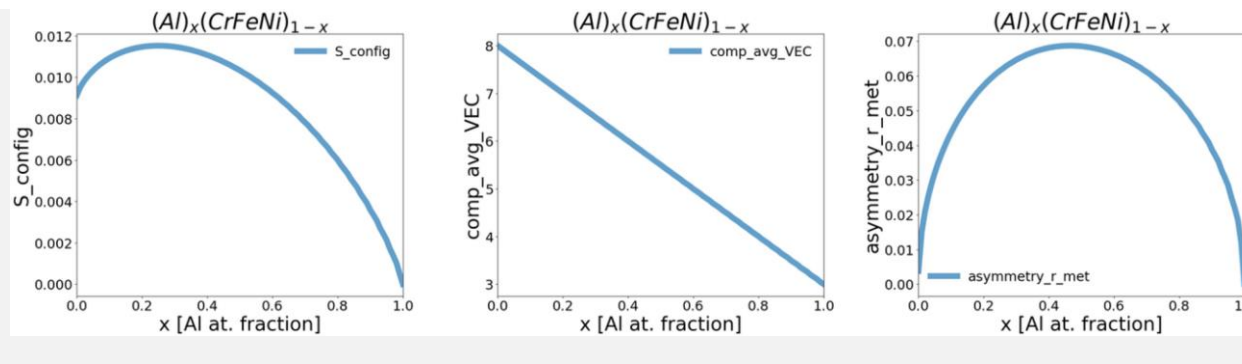
Terminal Output

```
-----
Creating alloys for (Al)-(CrFeNi)...  DONE.
-----

Calculating 'S_config' feature values...  DONE.
Calculating 'comp_avg_VEC' feature values...  DONE.
Calculating 'asymmetry_r_met' feature values...  DONE.

      alloy_name  xA[A=Al]  xB[B=CrFeNi]  xEl[Al]  \
alloy_id
1      Al0.0Cr0.333Fe0.333Ni0.333      0.00      1.00      0.00
2      Al0.01Cr0.33Fe0.33Ni0.33      0.01      0.99      0.01
3      Al0.02Cr0.327Fe0.327Ni0.327      0.02      0.98      0.02
4      Al0.03Cr0.323Fe0.323Ni0.323      0.03      0.97      0.03
5      Al0.04Cr0.32Fe0.32Ni0.32      0.04      0.96      0.04
...
97     Al0.96Cr0.013Fe0.013Ni0.013      0.96      0.04      0.96
98     Al0.97Cr0.01Fe0.01Ni0.01      0.97      0.03      0.97
99     Al0.98Cr0.007Fe0.007Ni0.007      0.98      0.02      0.98
100    Al0.99Cr0.003Fe0.003Ni0.003      0.99      0.01      0.99
101    Al1.0Cr0.0Fe0.0Ni0.0      1.00      0.00      1.00

      xEl[Cr]  xEl[Fe]  xEl[Ni]  S_config  comp_avg_VEC  asymmetry_r_met
alloy_id
1      0.333      0.333      0.333  0.009134      7.9992      0.003783
2      0.330      0.330      0.330  0.009508      7.9500      0.015086
3      0.327      0.327      0.327  0.009766      7.9008      0.020863
4      0.323      0.323      0.323  0.009980      7.8492      0.025253
5      0.320      0.320      0.320  0.010165      7.8000      0.028885
...
97     0.013      0.013      0.013  0.001726      3.1950      0.024953
98     0.010      0.010      0.010  0.001394      3.1500      0.021962
99     0.007      0.007      0.007  0.001039      3.1050      0.018438
100    0.003      0.003      0.003  0.000509      3.0450      0.012126
101    0.000      0.000      0.000  0.000000      3.0000      0.000000
```



`mapal.map_features.ternary(feats, A, B, C, dc=0.01, colorscale="Viridis")`

maps features to composition variations in a ternary system

Args:

feats: list

list of features to be mapped

A: str

component A e.g. "Al" or "Al0.5Ti"

B: str

component B e.g. "Al" or "Al0.5Ti"

C: str

component C e.g. "Al" or "Al0.5Ti"

dc: float (optional; default = 0.01; range (0,1))

concentration step size (in atomic fraction)

colorscale: str (optional; default = "Viridis")

colorscale used for feature contours (for options, refer to: <https://plotly.com/python/builtin-colorscales/>)

Returns:

figs: list of matplotlib figure objects

list containing ternary contour figures of all features

df_alloys: pandas dataframe

dataframe with all alloys and corresponding feature values

Examples

```
>>> # Mapping features to composition variations in ternary system
>>> from mapal import map_features;
>>> feats = ["S_config", ("comp_avg", "VEC"), ("asymmetry", "r_met")];
>>> A, B, C = "Al", "Ti", "CrFeNi";
>>> figs, df_alloy = map_features.ternary(feats, A, B, C, dc=0.01);
>>> print(df_alloy);

>>> # Saving a particular figure
>>> fig1_savename = f"{A}-{B}-{C}-S_config.svg";
>>> figs[0].write_image(fig1_savename);

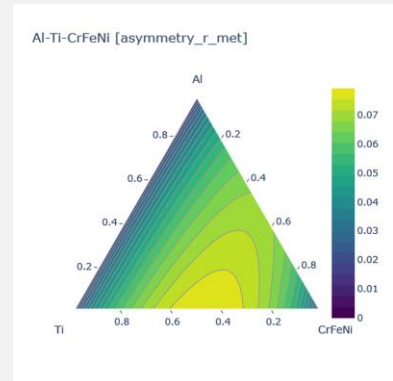
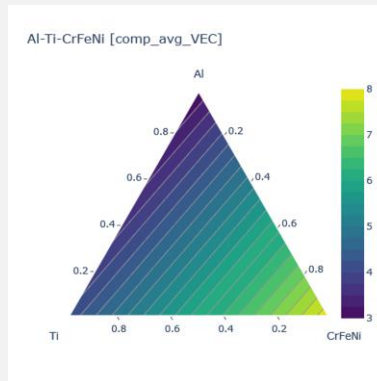
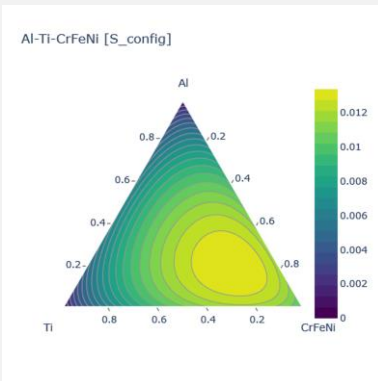
>>> # Saving the data
>>> df_savename = f"{A}-{B}-{C}-ternaryFeats.csv";
>>> df_alloy.to_csv(df_savename);
```

Terminal Output

Creating alloys for (Al)-(Ti)-(CrFeNi) ternary system... DONE.

Calculating 'S_config' feature values... DONE.
Creating contour plot...
Contour plot for S_config ready.
Calculating 'comp_avg_VEC' feature values... DONE.
Creating contour plot...
Contour plot for comp_avg_VEC ready.
Calculating 'asymmetry_r_met' feature values... DONE.
Creating contour plot...
Contour plot for asymmetry_r_met ready.

			alloy_name	Al	Ti	CrFeNi
S_config	comp_avg_VEC	asymmetry_r_met				
0	Al0.0Ti0.0Cr0.3333333333333333	Fe0.3333333333333333	...	0.00	0.00	1.00
0.009134	8.00	0.003781				
1	Al0.0Ti0.01Cr0.3299999999999999	Fe0.3299999999999999	...	0.00	0.01	0.99
0.009508	7.96	0.017407				
2	Al0.0Ti0.02Cr0.3266666666666666	Fe0.3266666666666666	...	0.00	0.02	0.98
0.009766	7.92	0.024165				
3	Al0.0Ti0.03Cr0.3233333333333333	Fe0.3233333333333333	...	0.00	0.03	0.97
0.009980	7.88	0.029278				
4	Al0.0Ti0.04Cr0.3199999999999999	Fe0.3199999999999999	...	0.00	0.04	0.96
0.010165	7.84	0.033508				
...			...	...	...	...
...	...	...				
5146	Al0.98Ti0.01Cr0.0033333333333333	Fe0.0033333333333333	...	0.98	0.01	0.01
0.001022	3.06	0.012965				
5147		Al0.98Ti0.02Cr0.0Fe0.0Ni0.0	...	0.98	0.02	0.00
0.000815	3.02	0.002936				
5148	Al0.99Ti0.0Cr0.0033333333333333	Fe0.0033333333333333	...	0.99	0.00	0.01
0.000557	3.05	0.012777				
5149		Al0.99Ti0.01Cr0.0Fe0.0Ni0.0	...	0.99	0.01	0.00
0.000466	3.01	0.002087				
5150		Al1.0Ti0.0Cr0.0Fe0.0Ni0.0	...	1.00	0.00	0.00
0.000000	3.00	0.000000				



`mapal.map_features.ternary1Cfixed(feats, A, B, C, Cfix, dc=0.01, cAmin=0, cAmax=1, nCols=4)`

maps features to composition variations in a ternary system where concentration of C component is fixed

Args:

feats: list

list of features to be mapped

A: str

component A e.g. "Al" or "Al0.5Ti"

B: str

component B e.g. "Al" or "Al0.5Ti"

C: str

component C e.g. "Al" or "Al0.5Ti"

Cfix: float

fixed concentration (atomic fraction) of component C

dc: float (optional; default = 0.01; range (0,1))

concentration step size (in atomic fraction)

cAmin: float (optional; default=0.0)

min concentration of component A (in atomic fraction)

cAmax: float (optional; default = (1-Cfix); range (0,1))

max concentration of component A; if not specified, it is set to max possible value i.e. (1-Cfix)

nCols: int

max no. of columns in final figure

Returns:

fig: matplotlib figure object

figure containing plots of all features

df_alloys: pandas dataframe

dataframe with all alloys and corresponding feature values

Examples

```
>>> # Mapping features to composition variations in ternary system where
>>> # concentration of C component is fixed
>>> from mapal import map_features;
>>> feats = ["S_config", ("comp_avg", "VEC"), ("asymmetry", "r_met")];
>>> A, B, C = "Al", "Ti", "CrFeNi";
>>> fig, df_alloy = map_features.ternary1Cfixed(feats, A, B, C, Cfix=0.6,
>>> # dc=0.01);
>>> print(df_alloy);

>>> # Saving the figure image
>>> fig_savename = "%s-%s-%s-ternary1Cfixed.png" % (A, B, C);
>>> fig.savefig(fig_savename, facecolor="white", edgecolor='none');

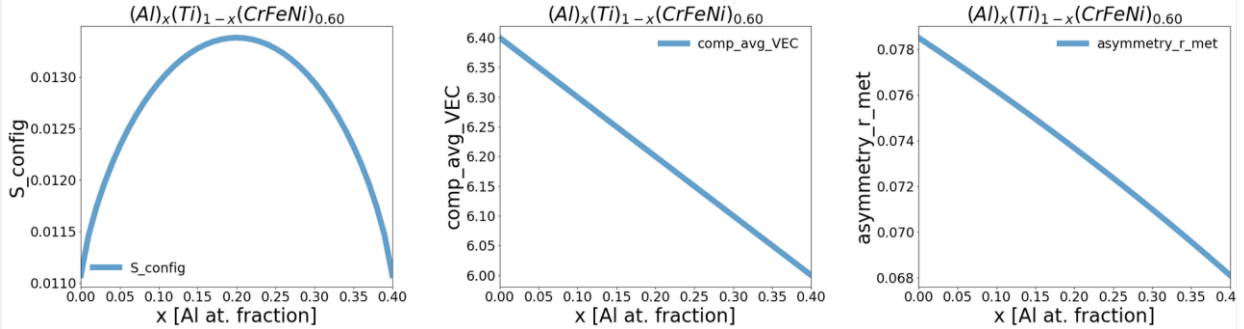
>>> # Saving the data
>>> df_savename = "%s-%s-%s-ternary1Cfixed.csv" % (A, B, C);
>>> df_alloy.to_csv(df_savename);
```

Terminal Output

```
-----
Creating alloys for (Al)-(Ti)-(CrFeNi) ternary system... DONE.
```

Calculating 'S_config' feature values... DONE.
Calculating 'comp_avg_VEC' feature values... DONE.
Calculating 'asymmetry_r_met' feature values... DONE.

	alloy_name	Al	Ti	CrFeNi
S_config	comp_avg_VEC	asymmetry_r_met		
0	Al0.0Ti0.4Cr0.19999999999999998Fe0.1999999999...	0.00	0.40	0.6
0.011076	6.40	0.078510		
1	Al0.01Ti0.39Cr0.19999999999999998Fe0.199999999...	0.01	0.39	0.6
0.011465	6.39	0.078284		
2	Al0.02Ti0.38Cr0.19999999999999998Fe0.199999999...	0.02	0.38	0.6
0.011736	6.38	0.078056		
3	Al0.03Ti0.37Cr0.19999999999999998Fe0.199999999...	0.03	0.37	0.6
0.011962	6.37	0.077826		
4	Al0.04Ti0.36Cr0.19999999999999998Fe0.199999999...	0.04	0.36	0.6
0.012157	6.36	0.077595		
5	Al0.05Ti0.35Cr0.19999999999999998Fe0.199999999...	0.05	0.35	0.6
0.012329	6.35	0.077362		
.....				
39	Al0.39Ti0.01Cr0.19999999999999998Fe0.199999999...	0.39	0.01	0.6
0.011465	6.01	0.068397		
40	Al0.4Ti0.0Cr0.19999999999999998Fe0.199999999...	0.40	0.00	0.6
0.011076	6.00	0.068098		



4.7. *mapal.map_ML_predictions*

Package to map alloy features over continuous compositional spaces in binary, ternary and multicomponent systems

Package requirements:

numpy, pandas, tqdm, matplotlib, plotly, scikit-image

List of functions:

`mapal.map_ML_predictions.HV_binary`

`mapal.map_ML_predictions.fPhase_binary`

`mapal.map_ML_predictions.HV_ternary`

`mapal.map_ML_predictions.fPhase_ternary`

`mapal.map_ML_predictions.HV_ternary1Cfixed`

`mapal.map_ML_predictions.fPhase_ternary1Cfixed`

`mapal.map_ML_predictions.HV_binary(A, B, dc=0.01, cAmin=0, cAmax=1, mod_key="M1")`
maps ML predicted hardness to composition variations in a binary system

Args:

A: str

component A e.g. "Al" or "Al0.5Ti"

B: str

component B e.g. "Al" or "Al0.5Ti"

dc: float (optional; default = 0.01; range (0,1))

concentration step size (in atomic fraction)

cAmin: float (optional, default=0.0)

min concentration of component A (in atomic fraction)

cAmax: float (optional; default = 1.0; range (0,1))

max concentration of component A (in atomic fraction)

mod_key: str (optional, default="M1")

model key to be used (currently only "M1" available for hardness prediction)

Returns:

fig: matplotlib figure object
figure with predicted hardness plot
df_alloys: pandas dataframe
dataframe with all alloys and corresponding hardness prediction

Examples

```
>>> # Mapping hardness prediction to composition variations in a binary
system
>>> from mapal import map_ML_predictions;

>>> A, B = "Al", "CrFeNi";
>>> fig, df_alloy = map_ML_predictions.HV_binary(A, B, dc=0.01, cAmin=0,
cAmax=0.4);
>>> print(df_alloy);

>>> # Saving the figure image
>>> fig_savename = "(%s)x-(%s)-binary-HVPredicted.png" % (A, B);
>>> fig.savefig(fig_savename, facecolor="white", edgecolor="none");

>>> # Saving the data
>>> df_savename = "(%s)x-(%s)-binary-HVPredicted.csv" % (A, B);
>>> df_alloy.to_csv(df_savename);
```

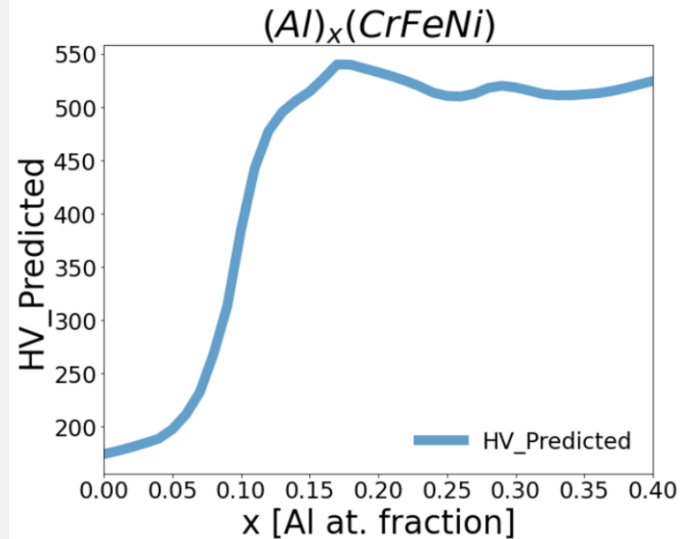
Terminal Output

```
-----
Creating alloys for (Al)-(CrFeNi)... DONE.
-----

--- Pre-trained model loaded (M1: Vickers hardness prediction) ---
Creating alloy features... DONE.
Normalizing features... DONE.
Predicting vickers hardness... DONE.
-----
```

	alloy_name	xA[A=Al]	xB[B=CrFeNi]	xEl[Al]	\\
alloy_id					
1	Al0.0Cr0.333Fe0.333Ni0.333	0.00	1.00	0.00	
2	Al0.01Cr0.33Fe0.33Ni0.33	0.01	0.99	0.01	
3	Al0.02Cr0.327Fe0.327Ni0.327	0.02	0.98	0.02	
4	Al0.03Cr0.323Fe0.323Ni0.323	0.03	0.97	0.03	
....					
38	Al0.37Cr0.21Fe0.21Ni0.21	0.37	0.63	0.37	
39	Al0.38Cr0.207Fe0.207Ni0.207	0.38	0.62	0.38	
40	Al0.39Cr0.203Fe0.203Ni0.203	0.39	0.61	0.39	
41	Al0.4Cr0.2Fe0.2Ni0.2	0.40	0.60	0.40	

	xEl[Cr]	xEl[Fe]	xEl[Ni]	HV_predicted
alloy_id				
1	0.333	0.333	0.333	173.646406
2	0.330	0.330	0.330	175.956235
3	0.327	0.327	0.327	178.754301
4	0.323	0.323	0.323	183.984500
....				
38	0.210	0.210	0.210	511.875329
39	0.207	0.207	0.207	514.187304
40	0.203	0.203	0.203	518.638954
41	0.200	0.200	0.200	521.305054



```
mapal.map_ML_predictions.fPhase_binary(A, B, dc=0.01, cAmin=0, cAmax=1,
mod_key="M2")
```

maps ML predicted phase fractions for FCC, BCC & Intermetallic phases to composition variations in a binary system

Args:

A: str

component A e.g. "Al" or "Al0.5Ti"

B: str

component B e.g. "Al" or "Al0.5Ti"

dc: float (optional; default = 0.01; range (0,1))

concentration step size (in atomic fraction)

cAmin: float (optional, default=0.0)

min concentration of component A (in atomic fraction)

cAmax: float (optional; default = 1.0; range (0,1))

max concentration of component A (in atomic fraction)

mod_key: str (optional, default="M2")

model key to be used (currently only "M2" available for phase fraction prediction)

Returns:

fig: matplotlib figure object

figure with predicted phase fraction plot

df_alloys: pandas dataframe

dataframe with all alloys and corresponding phase fraction prediction

Examples

```
>>> # Mapping phase fraction prediction to composition variations in a
binary system
>>> from mapal import map_ML_predictions;

>>> A, B = "Al", "CrFeNi";
```

```
>>> fig, df_alloy = map_ML_predictions.fPhase_binary(A, B, dc=0.01, cAmin=0,
cAmax=0.4);
>>> print(df_alloy);

>>> # Saving the figure image
>>> fig_savename = "(%s)x-(%s)-binary-fPhasePredicted.png" % (A, B);
>>> fig.savefig(fig_savename, facecolor="white", edgecolor="none");

>>> # Saving the data
>>> df_savename = "(%s)x-(%s)-binary-fPhasePredicted.csv" % (A, B);
>>> df_alloy.to_csv(df_savename);
```

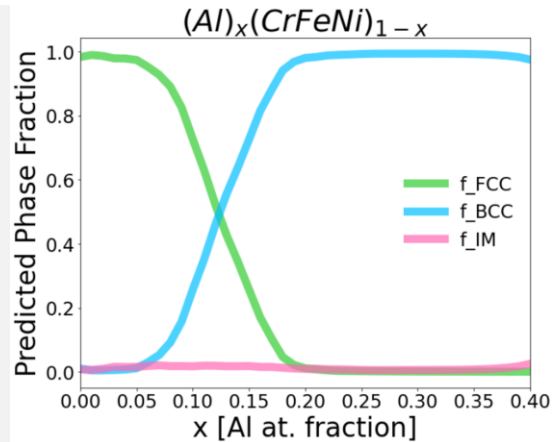
Terminal Output

```
-----
Creating alloys for (Al)-(CrFeNi)... DONE.
-----

--- Pre-trained model loaded (M2: FCC, BCC & Intermetallic phase fraction
prediction) ---
Creating alloy features... DONE.
Normalizing features... DONE.
Predicting phase fraction of FCC, BCC & Intermetallic phases... DONE.
-----
```

alloy_id	alloy_name	xA[A=Al]	xB[B=CrFeNi]	xEI[Al]	\\
1	Al0.0Cr0.333Fe0.333Ni0.333	0.00	1.00	0.00	
2	Al0.01Cr0.33Fe0.33Ni0.33	0.01	0.99	0.01	
3	Al0.02Cr0.327Fe0.327Ni0.327	0.02	0.98	0.02	
4	Al0.03Cr0.323Fe0.323Ni0.323	0.03	0.97	0.03	
....					
38	Al0.37Cr0.21Fe0.21Ni0.21	0.37	0.63	0.37	
39	Al0.38Cr0.207Fe0.207Ni0.207	0.38	0.62	0.38	
40	Al0.39Cr0.203Fe0.203Ni0.203	0.39	0.61	0.39	
41	Al0.4Cr0.2Fe0.2Ni0.2	0.40	0.60	0.40	

alloy_id	xEI[Cr]	xEI[Fe]	xEI[Ni]	f_FCC	f_BCC	f_IM
1	0.333	0.333	0.333	0.983243	0.010122	0.006635
2	0.330	0.330	0.330	0.989503	0.004395	0.006102
3	0.327	0.327	0.327	0.986379	0.004386	0.009236
4	0.323	0.323	0.323	0.978926	0.005742	0.015332
....						
38	0.210	0.210	0.210	0.000394	0.989520	0.010087
39	0.207	0.207	0.207	0.000368	0.987461	0.012171
40	0.203	0.203	0.203	0.000256	0.981058	0.018685
41	0.200	0.200	0.200	0.000225	0.974882	0.024894



```
mapal.map_ML_predictions.HV_ternary(A, B, C, dc=0.01, colorscale="Viridis",
mod_key="M1")
```

maps ML predicted hardness to composition variations in a ternary system

Args:

A: str

component A e.g. "Al" or "Al0.5Ti"

B: str

component B e.g. "Al" or "Al0.5Ti"

C: str

component C e.g. "Al" or "Al0.5Ti"

dc: float (optional; default = 0.01; range (0,1))

concentration step size (in atomic fraction)

colormap: str (optional; default = "Viridis")

colormap used for feature contours (for options, refer to:
<https://plotly.com/python/builtin-colorscales/>)

mod_key: str (optional, default="M1")

model key to be used (currently only "M1" available for hardness prediction)

Returns:

fig: plotly figure object

ternary contour plot of predicted hardness

df_alloys: pandas dataframe

dataframe with all alloys and corresponding hardness prediction

Examples

```
>>> # Mapping hardness prediction to composition variations in ternary
system
>>> from mapal import map_ML_predictions;

>>> A, B, C = "Al", "Ti", "CrFeNi";
>>> fig, df_alloy = map_ML_predictions.HV_ternary(A, B, C, dc=0.01,
colormap="Viridis");
>>> print(df_alloy);
```

```
>>> # Saving the figure image
>>> fig_savename = "(%s)-(%s)-(%s)-ternary-HVPredicted.png" % (A, B, C);
>>> fig.write_image(fig_savename);

>>> # Saving the data
>>> df_savename = "(%s)-(%s)-(%s)-ternary-HVPredicted.csv" % (A, B, C);
>>> df_alloy.to_csv(df_savename);
```

Terminal Output

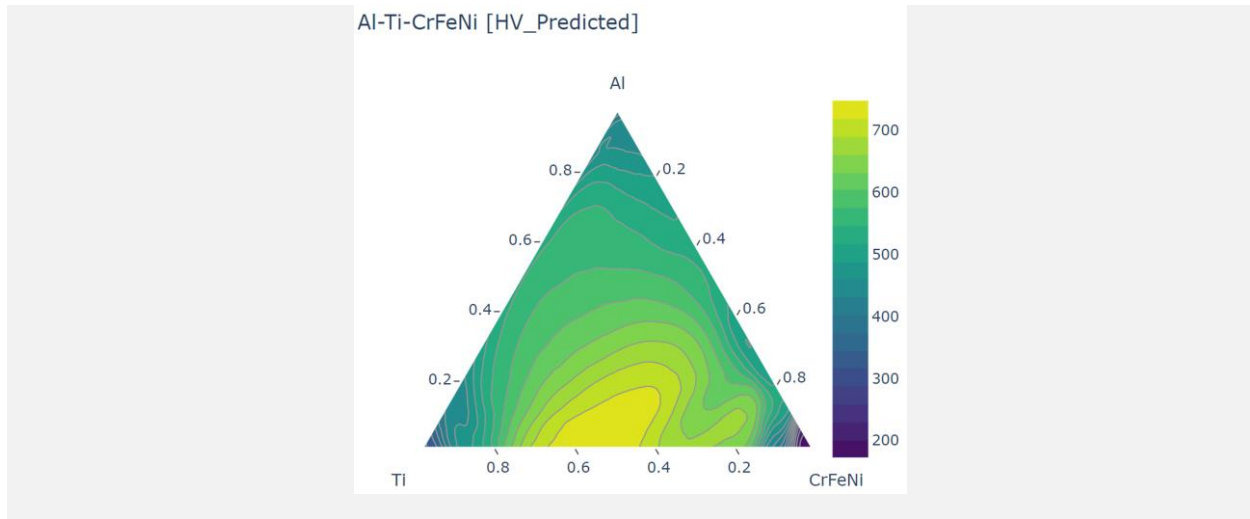
```
-----
Creating alloys for (Al)-(Ti)-(CrFeNi) ternary system... DONE.
-----
```

```
-----
--- Pre-trained model loaded (M1: Vickers hardness prediction) ---
Creating alloy features... DONE.
Normalizing features... DONE.
Predicting vickers hardness... DONE.
-----
```

```
Creating contour plot...
Contour plot ready.
```

	alloy_name	xA[A=Al]	xB[B=Ti]	xC[C=CrFeNi]
\\				
alloy_id				
1	Al0.0Cr0.333Fe0.333Ni0.333Ti0.0	0.00	0.00	1.00
2	Al0.0Cr0.33Fe0.33Ni0.33Ti0.01	0.00	0.01	0.99
3	Al0.0Cr0.327Fe0.327Ni0.327Ti0.02	0.00	0.02	0.98
4	Al0.0Cr0.323Fe0.323Ni0.323Ti0.03	0.00	0.03	0.97
5	Al0.0Cr0.32Fe0.32Ni0.32Ti0.04	0.00	0.04	0.96
...	...	...	...	...
5147	Al0.98Cr0.003Fe0.003Ni0.003Ti0.01	0.98	0.01	0.01
5148	Al0.98Cr0.0Fe0.0Ni0.0Ti0.02	0.98	0.02	0.00
5149	Al0.99Cr0.003Fe0.003Ni0.003Ti0.0	0.99	0.00	0.01
5150	Al0.99Cr0.0Fe0.0Ni0.0Ti0.01	0.99	0.01	0.00
5151	Al1.0Cr0.0Fe0.0Ni0.0Ti0.0	1.00	0.00	0.00

	xEl[Al]	xEl[Cr]	xEl[Fe]	xEl[Ni]	xEl[Ti]	HV_predicted
alloy_id						
1	0.00	0.333	0.333	0.333	0.00	173.646406
2	0.00	0.330	0.330	0.330	0.01	172.355134
3	0.00	0.327	0.327	0.327	0.02	171.260685
4	0.00	0.323	0.323	0.323	0.03	177.471294
5	0.00	0.320	0.320	0.320	0.04	191.131356
...	...	...	...	...	...	...
5147	0.98	0.003	0.003	0.003	0.01	385.394664
5148	0.98	0.000	0.000	0.000	0.02	340.400250
5149	0.99	0.003	0.003	0.003	0.00	375.557719
5150	0.99	0.000	0.000	0.000	0.01	325.524529
5151	1.00	0.000	0.000	0.000	0.00	320.955980



```
mapal.map_ML_predictions.fPhase_ternary(A, B, C, dc=0.01, colorscale="Viridis",
mod_key="M2")
```

maps ML predicted phase fractions of FCC, BCC and Intermetallic phases to composition variations in a ternary system

Args:

A: str

component A e.g. "Al" or "Al0.5Ti"

B: str

component B e.g. "Al" or "Al0.5Ti"

C: str

component C e.g. "Al" or "Al0.5Ti"

dc: float (optional; default = 0.01; range (0,1))

concentration step size (in atomic fraction)

colorscale: str (optional; default = "Viridis")

colorscale used for feature contours (for options, refer to: <https://plotly.com/python/builtin-colorscales/>)

mod key: str (optional, default="M2")

model key to be used (currently only "M2" available for hardness prediction)

Returns:

figs_dict: dict (contains plotly figure objects)

dictionary with ternary contour plots of predicted phase fractions (keys: 'f_FCC', 'f_BCC', 'f_IM')

df_alloys: pandas dataframe

dataframe with all alloys and corresponding phase fraction prediction

Examples

```
>>> # Mapping phase fraction predictions to composition variations in a
ternary system
>>> from mapal import map_ML_predictions;

>>> A, B, C = "Al", "Ti", "CrFeNi";
```

```
>>> figs_dict, df_alloy = map_ML_predictions.fPhase_ternary(A, B, C, dc=0.1,
colorscale="Viridis");
>>> print(df_alloy);

>>> # Saving the figure images
>>> for phase in list(figs_dict.keys()):
    fig_savename = "(%s)-(%s)-(%s)-ternary-%s-Predicted.svg" % (A, B,
C, phase);
    figs_dict[phase].write_image(fig_savename);

>>> # Saving the data
>>> df_savename = "(%s)-(%s)-(%s)-ternary-fPhasePredicted.csv" % (A, B, C);
>>> df_alloy.to_csv(df_savename);
```

Terminal Output

```
-----
Creating alloys for (Al)-(Ti)-(CrFeNi) ternary system... DONE.
-----

--- Pre-trained model loaded (M2: FCC, BCC & Intermetallic phase fraction
prediction) ---
Creating alloy features... DONE.
Normalizing features... DONE.
Predicting phase fraction of FCC, BCC & Intermetallic phases... DONE.
-----

Creating contour plot for 'f_FCC'...
Contour plot for 'f_FCC' ready.
Creating contour plot for 'f_BCC'...
Contour plot for 'f_BCC' ready.
Creating contour plot for 'f_IM'...
Contour plot for 'f_IM' ready.

\\
alloy_name  xA[A=Al]  xB[B=Ti]  xC[C=CrFeNi]
\\
alloy_id
1  Al0.0Cr0.333Fe0.333Ni0.333Ti0.0  0.0  0.0  1.0
2  Al0.0Cr0.3Fe0.3Ni0.3Ti0.1  0.0  0.1  0.9
3  Al0.0Cr0.267Fe0.267Ni0.267Ti0.2  0.0  0.2  0.8
4  Al0.0Cr0.233Fe0.233Ni0.233Ti0.3  0.0  0.3  0.7
5  Al0.0Cr0.2Fe0.2Ni0.2Ti0.4  0.0  0.4  0.6
...
62  Al0.8Cr0.033Fe0.033Ni0.033Ti0.1  0.8  0.1  0.1
63  Al0.8Cr0.0Fe0.0Ni0.0Ti0.2  0.8  0.2  0.0
64  Al0.9Cr0.033Fe0.033Ni0.033Ti0.0  0.9  0.0  0.1
65  Al0.9Cr0.0Fe0.0Ni0.0Ti0.1  0.9  0.1  0.0
66  Al1.0Cr0.0Fe0.0Ni0.0Ti0.0  1.0  0.0  0.0

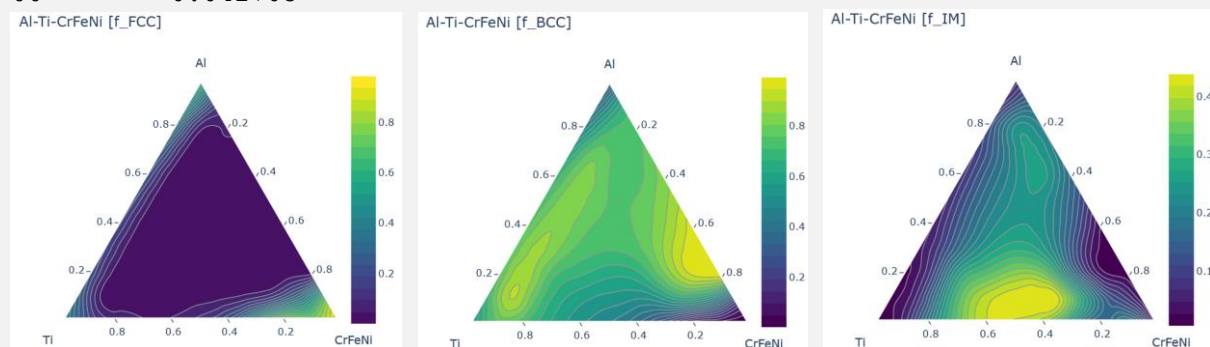
\\
alloy_id  xEl[Al]  xEl[Cr]  xEl[Fe]  xEl[Ni]  xEl[Ti]  f_FCC  f_BCC  \\
1  0.0  0.333  0.333  0.333  0.0  0.983243  0.010122
2  0.0  0.300  0.300  0.300  0.1  0.774746  0.000674
3  0.0  0.267  0.267  0.267  0.2  0.847159  0.010321
4  0.0  0.233  0.233  0.233  0.3  0.533836  0.207075
5  0.0  0.200  0.200  0.200  0.4  0.241311  0.380139
...
62  0.8  0.033  0.033  0.033  0.1  0.077278  0.690285
63  0.8  0.000  0.000  0.000  0.2  0.443441  0.454461
```

64	0.9	0.033	0.033	0.033	0.0	0.196938	0.649117
65	0.9	0.000	0.000	0.000	0.1	0.598845	0.340925
66	1.0	0.000	0.000	0.000	0.0	0.742300	0.214997

```

f_IM
alloy_id
1      0.006635
2      0.224580
3      0.142520
4      0.259089
5      0.378550
...
62     0.232437
63     0.102097
64     0.153945
65     0.060230
66     0.042703

```



```
mapal.map_ML_predictions.HV_ternary1Cfixed(A, B, C, Cfix, dc=0.01, cAmin=0,
cAmax="auto", mod_key="M1")
```

maps ML predicted hardness to composition variations in a ternary system where concentration of C component is fixed

Args:

A: str

component A e.g. "Al" or "Al0.5Ti"

B: str

component B e.g. "Al" or "Al0.5Ti"

C: str

component C e.g. "Al" or "Al0.5Ti"

Cfix: float

fixed concentration (atomic fraction) of component C

dc: float (optional; default = 0.01; range (0,1))

concentration step size (in atomic fraction)

cAmin: float (optional, default=0.0)

min concentration of component A (in atomic fraction)

cAmax: float (optional; default = (1-Cfix); range (0,1))

max concentration of component A; if not specified, it is set to max possible value
i.e. (1-Cfix)
mod_key: str (optional, default="M1")
model key to be used (currently only "M1" available for hardness prediction)

Returns:

fig: matplotlib figure object
figure with predicted hardness plot
df_alloys: pandas dataframe
dataframe with all alloys and corresponding hardness prediction

Examples

```
>>> # Mapping hardness predictions to composition variations in ternary
system where concentration of C component is fixed
>>> from mapal import map_ML_predictions;

>>> A, B, C = "Al", "Ti", "CrFeNi";
>>> Cfix = 0.6;
>>> fig, df_alloy = map_ML_predictions.HV_ternary1Cfixed(A, B, C, Cfix,
dc=0.05, cAmin=0, cAmax="auto");
>>> print(df_alloy);

>>> # Saving the figure image
>>> fig_savename = "(%s)-(%s)-(%s)-ternary1Cfixed-HVPredicted.png" % (A, B,
C);
>>> fig.savefig(fig_savename, facecolor="white", edgecolor="none");

>>> # Saving the data
>>> df_savename = "(%s)-(%s)-(%s)-ternary1Cfixed-HVPredicted.csv" % (A, B,
C);
>>> df_alloy.to_csv(df_savename);
```

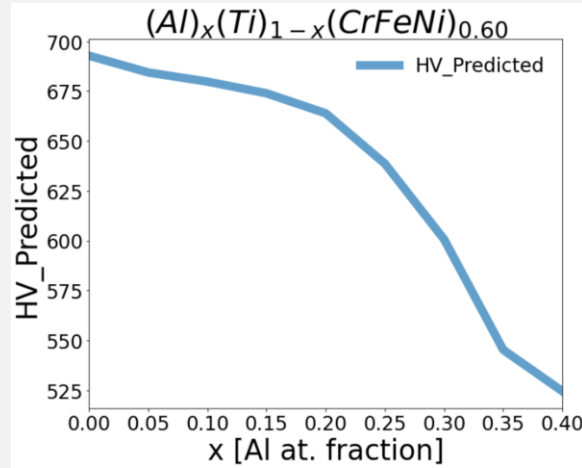
Terminal Output

```
Concentration of (CrFeNi) fixed at '0.600'
Creating alloys for (Al)-(Ti)-(CrFeNi) ternary system... DONE.
-----
--- Pre-trained model loaded (M1: Vickers hardness prediction) ---
Creating alloy features... DONE.
Normalizing features... DONE.
Predicting vickers hardness... DONE.
-----

      alloy_name  xA[A=Al]  xB[B=Ti]  xC[C=CrFeNi]  \
alloy_id
1      Al0.0Cr0.2Fe0.2Ni0.2Ti0.4      0.00      0.40      0.6
2      Al0.05Cr0.2Fe0.2Ni0.2Ti0.35      0.05      0.35      0.6
3      Al0.1Cr0.2Fe0.2Ni0.2Ti0.3      0.10      0.30      0.6
4      Al0.15Cr0.2Fe0.2Ni0.2Ti0.25      0.15      0.25      0.6
5      Al0.2Cr0.2Fe0.2Ni0.2Ti0.2      0.20      0.20      0.6
6      Al0.25Cr0.2Fe0.2Ni0.2Ti0.15      0.25      0.15      0.6
7      Al0.3Cr0.2Fe0.2Ni0.2Ti0.1      0.30      0.10      0.6
8      Al0.35Cr0.2Fe0.2Ni0.2Ti0.05      0.35      0.05      0.6
9      Al0.4Cr0.2Fe0.2Ni0.2Ti0.0      0.40      0.00      0.6

      xEl[Al]  xEl[Cr]  xEl[Fe]  xEl[Ni]  xEl[Ti]  HV_predicted
```

alloy_id						
1	0.00	0.2	0.2	0.2	0.40	687.501362
2	0.05	0.2	0.2	0.2	0.35	679.590549
3	0.10	0.2	0.2	0.2	0.30	675.075698
4	0.15	0.2	0.2	0.2	0.25	669.130393
5	0.20	0.2	0.2	0.2	0.20	659.152606
6	0.25	0.2	0.2	0.2	0.15	634.617526
7	0.30	0.2	0.2	0.2	0.10	596.902462
8	0.35	0.2	0.2	0.2	0.05	542.150587
9	0.40	0.2	0.2	0.2	0.00	521.305054



`mapal.map_ML_predictions.fPhase_ternary1Cfixed(A, B, C, Cfix, dc=0.01, cAmin=0, cAmax="auto", mod_key="M2")`

maps ML predicted phase fractions for FCC, BCC & Intermetallic phases to composition variations in a ternary system where concentration of C component is fixed

Args:

A: str

component A e.g. "Al" or "Al0.5Ti"

B: str

component B e.g. "Al" or "Al0.5Ti"

C: str

component C e.g. "Al" or "Al0.5Ti"

Cfix: float

fixed concentration (atomic fraction) of component C

dc: float (optional; default = 0.01; range (0,1))

concentration step size (in atomic fraction)

cAmin: float (optional, default=0.0)

min concentration of component A (in atomic fraction)

cAmax: float (optional; default = (1-Cfix); range (0,1))

max concentration of component A; if not specified, it is set to max possible value i.e. (1-Cfix)

mod_key: str (optional, default="M2")

model key to be used (currently only "M2" available for phase fraction predictions)

Returns:

fig: matplotlib figure object

figure with predicted phase fraction plot

df_alloys: pandas dataframe

dataframe with all alloys and corresponding phase fraction prediction

Examples

```
>>> # Mapping phase fraction predictions to composition variations in
ternary system where concentration of C component is fixed
>>> from mapal import map_ML_predictions;

>>> A, B, C = "Al", "Ti", "CrFeNi";
>>> Cfix = 0.6;
>>> fig, df_alloy = map_ML_predictions.fPhase_ternary1Cfixed(A, B, C, Cfix,
dc=0.05, cAmin=0, cAmax="auto");
>>> print(df_alloy);

>>> # Saving the figure image
>>> fig_savename = "(%s)-(%s)-(%s)-ternary1Cfixed-fPhasePredicted.png" % (A,
B, C);
>>> fig.savefig(fig_savename, facecolor="white", edgecolor="none");

>>> # Saving the data
>>> df_savename = "(%s)-(%s)-(%s)-ternary1Cfixed-fPhasePredicted.csv" % (A,
B, C);
>>> df_alloy.to_csv(df_savename);
```

Terminal Output

```
Concentration of (CrFeNi) fixed at '0.600'
Creating alloys for (Al)-(Ti)-(CrFeNi) ternary system... DONE.
-----
--- Pre-trained model loaded (M2: FCC, BCC & Intermetallic phase fraction
prediction) ---
Creating alloy features... DONE.
Normalizing features... DONE.
Predicting phase fraction of FCC, BCC & Intermetallic phases... DONE.
-----
```

	alloy_name	xA[A=Al]	xB[B=Ti]	xC[C=CrFeNi]	\\
alloy_id					
1	Al0.0Cr0.2Fe0.2Ni0.2Ti0.4	0.00	0.40	0.6	
2	Al0.05Cr0.2Fe0.2Ni0.2Ti0.35	0.05	0.35	0.6	
3	Al0.1Cr0.2Fe0.2Ni0.2Ti0.3	0.10	0.30	0.6	
4	Al0.15Cr0.2Fe0.2Ni0.2Ti0.25	0.15	0.25	0.6	
5	Al0.2Cr0.2Fe0.2Ni0.2Ti0.2	0.20	0.20	0.6	
6	Al0.25Cr0.2Fe0.2Ni0.2Ti0.15	0.25	0.15	0.6	
7	Al0.3Cr0.2Fe0.2Ni0.2Ti0.1	0.30	0.10	0.6	
8	Al0.35Cr0.2Fe0.2Ni0.2Ti0.05	0.35	0.05	0.6	
9	Al0.4Cr0.2Fe0.2Ni0.2Ti0.0	0.40	0.00	0.6	

	xEl[Al]	xEl[Cr]	xEl[Fe]	xEl[Ni]	xEl[Ti]	f_FCC	f_BCC	\\
alloy_id								
1	0.00	0.2	0.2	0.2	0.40	0.241311	0.380139	

2	0.05	0.2	0.2	0.2	0.35	0.117014	0.450446
3	0.10	0.2	0.2	0.2	0.30	0.054936	0.522614
4	0.15	0.2	0.2	0.2	0.25	0.021303	0.644821
5	0.20	0.2	0.2	0.2	0.20	0.004536	0.803616
6	0.25	0.2	0.2	0.2	0.15	0.001100	0.921701
7	0.30	0.2	0.2	0.2	0.10	0.000761	0.934165
8	0.35	0.2	0.2	0.2	0.05	0.000403	0.947957
9	0.40	0.2	0.2	0.2	0.00	0.000225	0.974882

	f_IM
alloy_id	
1	0.378550
2	0.432540
3	0.422450
4	0.333875
5	0.191849
6	0.077199
7	0.065073
8	0.051640
9	0.024894

